

```

!pip install planetary-computer pystac-client rasterio geopandas matplotlib contextily numpy pandas scikit-learn shapely pyproj

import warnings
warnings.filterwarnings("ignore")

import os
import json
import math
import hashlib
from datetime import datetime

import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt

from shapely.geometry import Polygon
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

import rasterio
from rasterio.mask import mask
from rasterio.enums import Resampling
from rasterio.warp import reproject
import contextily as ctx
import planetary_computer
from pystac_client import Client
from matplotlib.lines import Line2D

OUTPUT_DIR = "Obuassi_GDI_Analysis "
os.makedirs(OUTPUT_DIR, exist_ok=True)

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

AOI_NAME = "Obuasi"
AOI_BBOX = [-1.78, 6.17, -1.63, 6.32] # xmin/lon, ymin/lat, xmax/lon, ymax/lat

TIME_LABEL_1 = "Jan_2026"
TIME_LABEL_2 = "Apr_2026"
DATE_RANGE_1 = "2026-01-01/2026-01-31"
DATE_RANGE_2 = "2026-02-01/2026-04-30"

MAX_ITEMS = 15
MAX_CLOUD = 20

MAX_CANDIDATE_POINTS = 1200
MIN_PIXEL_STRIDE = 3

ADAPTIVE_PERCENTILE = 75
SENSITIVITY_PERCENTILES = [70, 75, 80]
PERSISTENCE_THRESHOLD = 0.40
CANDIDATE_SCORE_THRESHOLD = 0.4

DBSCAN_EPS_DEGREES = 0.003
DBSCAN_MIN_SAMPLES = 4

LOW_MOD_Q = 0.50
MOD_HIGH_Q = 0.75
HIGH_CRIT_Q = 0.90
LOW_CONFIDENCE_THRESHOLD = 0.30

# Legal concession remains external governance layer.
CONCESSION_FILE = "Ghana_Mining_Concessions.shp"
USE_DEMO_LEGAL_ZONE_IF_MISSING = True
SHOW_LEGAL_CONCESSION_ON_MAP = False # Recommended False if demo concession is used.

# Optional land-cover mask. Disabled unless a real raster is supplied.
LANDCOVER_FILE = "landcover_mask.tif"
USE_LANDCOVER_MASK = False
EXCLUDE_LANDCOVER_CLASSES = []

SHOW_HIGH_RISK_LABELS_ON_MAP = True

```

```

ANALYSIS_RESULT_ID = "ipfs://analysis_obuasi_final_gdi_risk_outputs_2026"
SOURCE_NAME = "Sentinel2_L2A_MicrosoftPlanetaryComputer"

print("Configuration loaded.")

def minmax_series(series: pd.Series) -> pd.Series:
    s = pd.Series(series).replace([np.inf, -np.inf], np.nan)
    smin, smax = s.min(), s.max()
    if pd.isna(smin) or pd.isna(smax) or smax == smin:
        return pd.Series(np.zeros(len(s)), index=s.index)
    return (s - smin) / (smax - smin)

def minmax_array(arr: np.ndarray) -> np.ndarray:
    out = np.full(arr.shape, np.nan, dtype="float32")
    m = np.isfinite(arr)
    if not m.any():
        return out
    vals = arr[m]
    amin, amax = vals.min(), vals.max()
    if amax == amin:
        out[m] = 0.0
    else:
        out[m] = (vals - amin) / (amax - amin)
    return out

def compute_hash(data_dict: dict) -> str:
    payload = json.dumps(data_dict, sort_keys=True, default=str).encode("utf-8")
    return hashlib.sha256(payload).hexdigest()

def create_aoi_polygon(bbox):
    xmin, ymin, xmax, ymax = bbox
    return Polygon([(xmin, ymin), (xmin, ymax), (xmax, ymax), (xmax, ymin), (xmin, ymin)])

def bbox_to_geojson(bbox):
    xmin, ymin, xmax, ymax = bbox
    return {
        "type": "Polygon",
        "coordinates": [[[xmin, ymin], [xmin, ymax], [xmax, ymax], [xmax, ymin], [xmin, ymin]]]
    }

def scaled_coord(value: float) -> int:
    return int(round(float(value) * 10000))

def scaled_gdi(value: float) -> int:
    return int(round(float(value) * 10000))

def classify_gdi_from_thresholds(gdi, low_mod, mod_high, high_crit):
    if gdi < low_mod:
        return "Low"
    if gdi < mod_high:
        return "Moderate"
    if gdi < high_crit:
        return "High"
    return "Critical"

def save_array_png(arr, title, outpath, cmap="viridis", vmin=None, vmax=None):
    plt.figure(figsize=(10, 8))
    plt.imshow(arr, cmap=cmap, vmin=vmin, vmax=vmax)
    plt.colorbar()
    plt.title(title)
    plt.axis("off")
    plt.tight_layout()
    plt.savefig(outpath, dpi=300, bbox_inches="tight")
    plt.close()

catalog = Client.open("https://planetarycomputer.microsoft.com/api/stac/v1")
print("Connected to Microsoft Planetary Computer.")

```

```

def load_concessions():
    if os.path.exists(CONCESSION_FILE):
        concessions = gpd.read_file(CONCESSION_FILE).to_crs("EPSG:4326")
        print(f"Loaded concession file with {len(concessions)} features.")
        return concessions

    print("Concession file not found.")
    if USE_DEMO_LEGAL_ZONE_IF_MISSING:
        demo_poly = Polygon([
            (-1.75, 6.20),
            (-1.75, 6.28),
            (-1.67, 6.28),
            (-1.67, 6.20),
            (-1.75, 6.20)
        ])
        concessions = gpd.GeoDataFrame(
            {"name": ["Demo_Obuasi_Legal_Zone"], "geometry": [demo_poly]},
            crs="EPSG:4326"
        )
        print("Using demo legal zone for external governance screening.")
        return concessions

    print("Proceeding without concessions; all points will be treated as unauthorized.")
    return gpd.GeoDataFrame(geometry=[], crs="EPSG:4326")

concessions = load_concessions()

def search_best_scene(date_range, label, aoi_geojson, max_items=MAX_ITEMS, max_cloud=MAX_CLOUD):
    print(f"Searching Sentinel-2 scene for {label}: {date_range}")
    search = catalog.search(
        collections=["sentinel-2-l2a"],
        intersects=aoi_geojson,
        datetime=date_range,
        query={"eo:cloud_cover": {"lt": max_cloud}},
        limit=max_items
    )
    items = list(search.items())
    if not items:
        raise RuntimeError(f"No suitable Sentinel-2 scenes found for {label}.")
    best = sorted(items, key=lambda x: x.properties.get("eo:cloud_cover", 999))[0]
    print(f"Selected {label}: {best.id}, cloud={best.properties.get('eo:cloud_cover')}")
    return best

def safe_read_band(item, band_name):
    signed = planetary_computer.sign(item)
    href = signed.assets[band_name].href
    return rasterio.open(href)

def read_clip_band(item, band_name, aoi_polygon):
    src = safe_read_band(item, band_name)
    aoi_gdf = gpd.GeoDataFrame({"geometry": [aoi_polygon]}, crs="EPSG:4326")
    aoi_proj = aoi_gdf.to_crs(src.crs)
    clipped, transform = mask(src, aoi_proj.geometry, crop=True, filled=True)
    arr = clipped[0].astype("float32")
    nodata = src.nodata
    if nodata is not None:
        arr[arr == nodata] = np.nan
    meta = src.meta.copy()
    meta.update({
        "height": arr.shape[0],
        "width": arr.shape[1],
        "transform": transform,
        "count": 1,
        "dtype": "float32",
        "crs": src.crs
    })
    src.close()
    return arr, meta

def resample_to_match(src_arr, src_meta, ref_meta):
    dst_arr = np.empty((ref_meta["height"], ref_meta["width"]), dtype="float32")
    reproject(

```

```

        source=src_arr,
        destination=dst_arr,
        src_transform=src_meta["transform"],
        src_crs=src_meta["crs"],
        dst_transform=ref_meta["transform"],
        dst_crs=ref_meta["crs"],
        resampling=Resampling.bilinear
    )
    return dst_arr

def align_to_reference(arr, meta, ref_meta):
    if (
        meta["height"] != ref_meta["height"] or
        meta["width"] != ref_meta["width"] or
        meta["transform"] != ref_meta["transform"] or
        meta["crs"] != ref_meta["crs"]
    ):
        return resample_to_match(arr, meta, ref_meta)
    return arr

def compute_ndvi(nir, red):
    arr = (nir - red) / (nir + red + 1e-10)
    arr[~np.isfinite(arr)] = np.nan
    return arr

def compute_bsi(red, nir, blue, swir):
    arr = ((swir + red) - (nir + blue)) / ((swir + red) + (nir + blue) + 1e-10)
    arr[~np.isfinite(arr)] = np.nan
    return arr

def compute_ndmi(nir, swir):
    arr = (nir - swir) / (nir + swir + 1e-10)
    arr[~np.isfinite(arr)] = np.nan
    return arr

def compute_mndwi(green, swir):
    arr = (green - swir) / (green + swir + 1e-10)
    arr[~np.isfinite(arr)] = np.nan
    return arr

def derive_adaptive_thresholds(ndvi_change, bsi_change, ndmi_change, mndwi_change, percentile):
    ndvi_loss = ndvi_change[np.isfinite(ndvi_change) & (ndvi_change < 0)]
    bsi_gain = bsi_change[np.isfinite(bsi_change) & (bsi_change > 0)]
    ndmi_loss = ndmi_change[np.isfinite(ndmi_change) & (ndmi_change < 0)]
    mndwi_dist = np.abs(mndwi_change[np.isfinite(mndwi_change)])

    ndvi_thr = -np.percentile(np.abs(ndvi_loss), percentile) if len(ndvi_loss) else -0.10
    bsi_thr = np.percentile(bsi_gain, percentile) if len(bsi_gain) else 0.05
    ndmi_thr = -np.percentile(np.abs(ndmi_loss), percentile) if len(ndmi_loss) else -0.08
    mndwi_thr = np.percentile(mndwi_dist, percentile) if len(mndwi_dist) else 0.05

    return float(ndvi_thr), float(bsi_thr), float(ndmi_thr), float(mndwi_thr)

def compute_persistence_score(ndvi_change, bsi_change, ndmi_change, mndwi_change):
    ndvi_loss_component = minmax_array(np.abs(np.where(np.isfinite(ndvi_change), np.minimum(ndvi_change, 0), np.nan)))
    bsi_gain_component = minmax_array(np.where(np.isfinite(bsi_change), np.maximum(bsi_change, 0), np.nan))
    ndmi_loss_component = minmax_array(np.abs(np.where(np.isfinite(ndmi_change), np.minimum(ndmi_change, 0), np.nan)))
    mndwi_dist_component = minmax_array(np.abs(mndwi_change))

    persistence = (ndvi_loss_component + bsi_gain_component + ndmi_loss_component + mndwi_dist_component) / 4.0
    persistence[~np.isfinite(persistence)] = np.nan
    return persistence

def extract_candidate_points(
    ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, meta, region_name,
    ndvi_thr, bsi_thr, ndmi_thr, mndwi_thr,
    persistence_threshold=PERSISTENCE_THRESHOLD,
    candidate_score_threshold=CANDIDATE_SCORE_THRESHOLD,
    max_points=MAX_CANDIDATE_POINTS,
    min_stride=MIN_PIXEL_STRIDE
):

```

```

ndvi_mask = np.isfinite(ndvi_change) & (ndvi_change < ndvi_thr)
bsi_mask = np.isfinite(bsi_change) & (bsi_change > bsi_thr)
ndmi_mask = np.isfinite(ndmi_change) & (ndmi_change < ndmi_thr)
mndwi_mask = np.isfinite(mndwi_change) & (np.abs(mndwi_change) > mndwi_thr)
persistence_mask = np.isfinite(persistence_score) & (persistence_score > persistence_threshold)

ndvi_component = minmax_array(np.abs(np.where(np.isfinite(ndvi_change), np.minimum(ndvi_change, 0), np.nan)))
bsi_component = minmax_array(np.where(np.isfinite(bsi_change), np.maximum(bsi_change, 0), np.nan))
ndmi_component = minmax_array(np.abs(np.where(np.isfinite(ndmi_change), np.minimum(ndmi_change, 0), np.nan)))
mndwi_component = minmax_array(np.abs(mndwi_change))
persistence_component = minmax_array(persistence_score)

candidate_score = np.nanmean(
    np.stack([
        ndvi_component,
        bsi_component,
        ndmi_component,
        mndwi_component,
        persistence_component
    ]),
    axis=0
)

candidate_mask = (
    ndvi_mask &
    bsi_mask &
    ndmi_mask &
    mndwi_mask &
    persistence_mask &
    np.isfinite(candidate_score) &
    (candidate_score > candidate_score_threshold)
)

rows, cols = np.where(candidate_mask)
if len(rows) == 0:
    empty = gpd.GeoDataFrame(
        columns=[
            "Region", "row", "col", "ndvi_change", "bsi_change", "ndmi_change", "mndwi_change",
            "persistence_score", "candidate_score", "geometry"
        ],
        geometry="geometry",
        crs=meta["crs"]
    )
    return empty.to_crs("EPSG:4326"), candidate_score, candidate_mask

stride = max(min_stride, math.ceil(len(rows) / max_points))
rows = rows[::stride]
cols = cols[::stride]

xs, ys, vals = [], [], []
transform = meta["transform"]
for r, c in zip(rows, cols):
    x, y = rasterio.transform.xy(transform, r, c, offset="center")
    xs.append(x)
    ys.append(y)
    vals.append({
        "Region": region_name,
        "row": int(r),
        "col": int(c),
        "ndvi_change": float(ndvi_change[r, c]),
        "bsi_change": float(bsi_change[r, c]),
        "ndmi_change": float(ndmi_change[r, c]),
        "mndwi_change": float(mndwi_change[r, c]),
        "persistence_score": float(persistence_score[r, c]),
        "candidate_score": float(candidate_score[r, c])
    })

df = pd.DataFrame(vals)
gdf = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(xs, ys), crs=meta["crs"]).to_crs("EPSG:4326")
return gdf, candidate_score, candidate_mask

def apply_optional_landcover_mask(candidates_gdf):
    stats = {
        "landcover_mask_used": False,
        "input_points": len(candidates_gdf),
        "removed_points": 0,
    }

```

```

        "remaining_points": len(candidates_gdf),
        "note": "Land-cover masking disabled or unavailable."
    }

if not USE_LANDCOVER_MASK or not os.path.exists(LANDCOVER_FILE) or len(EXCLUDE_LANDCOVER_CLASSES) == 0 or candidates_gdf.empty:
    return candidates_gdf, pd.DataFrame([stats])

try:
    with rasterio.open(LANDCOVER_FILE) as src:
        points_proj = candidates_gdf.to_crs(src.crs)
        values = []
        lc_arr = src.read(1)
        for geom in points_proj.geometry:
            row, col = src.index(geom.x, geom.y)
            if 0 <= row < src.height and 0 <= col < src.width:
                values.append(lc_arr[row, col])
            else:
                values.append(np.nan)

    out = candidates_gdf.copy()
    out["landcover_class"] = values
    before = len(out)
    out = out[~out["landcover_class"].isin(EXCLUDE_LANDCOVER_CLASSES)].copy()
    after = len(out)
    stats.update({
        "landcover_mask_used": True,
        "removed_points": before - after,
        "remaining_points": after,
        "note": f"Excluded classes: {EXCLUDE_LANDCOVER_CLASSES}"
    })
    return out, pd.DataFrame([stats])
except Exception as e:
    stats["note"] = f"Land-cover masking attempted but failed: {e}"
    return candidates_gdf, pd.DataFrame([stats])

def apply_legal_screening(mining_gdf, concessions_gdf, buffer_degrees=0.002):
    if mining_gdf.empty:
        return mining_gdf

    gdf = mining_gdf.copy()
    if concessions_gdf is None or concessions_gdf.empty:
        gdf["legal_status"] = "UNAUTHORIZED"
        return gdf

    concessions_local = concessions_gdf.to_crs(gdf.crs)
    buffer_local = concessions_local.copy()
    buffer_local["geometry"] = buffer_local.geometry.buffer(buffer_degrees)

    joined = gpd.sjoin(gdf, concessions_local, how="left", predicate="intersects")
    joined_buffer = gpd.sjoin(gdf, buffer_local, how="left", predicate="intersects")

    legal = pd.Series("UNAUTHORIZED", index=gdf.index)
    legal[joined.index[joined["index_right"].notna()]] = "LEGAL"
    boundary_idx = joined_buffer.index[joined_buffer["index_right"].notna()]
    legal[(legal == "UNAUTHORIZED") & (legal.index.isin(boundary_idx))] = "BOUNDARY"

    gdf["legal_status"] = legal.values
    return gdf

# =====
# PCA-GDI COMPUTATION
# =====
def compute_pca_weights(feature_df, feature_cols):
    X = feature_df[feature_cols].replace([np.inf, -np.inf], np.nan).fillna(0.0).values
    if len(feature_df) < 3:
        weights = np.ones(len(feature_cols)) / len(feature_cols)
        return weights, 0.0

    Xs = StandardScaler().fit_transform(X)
    pca = PCA(n_components=1, random_state=RANDOM_SEED)
    pca.fit(Xs)
    loadings = np.abs(pca.components_[0])
    weights = loadings / loadings.sum() if loadings.sum() != 0 else np.ones(len(feature_cols)) / len(feature_cols)
    explained = float(pca.explained_variance_ratio_[0])
    return weights, explained

```

```

def compute_gdi(candidate_points):
    if candidate_points.empty:
        return candidate_points, {}, pd.DataFrame()

    gdf = candidate_points.copy().reset_index(drop=True)

    coords = np.column_stack((gdf.geometry.x, gdf.geometry.y))
    db = DBSCAN(eps=DBSCAN_EPS_DEGREES, min_samples=DBSCAN_MIN_SAMPLES).fit(coords)
    gdf["cluster_label"] = db.labels_
    cluster_sizes = gdf["cluster_label"].value_counts().to_dict()
    gdf["cluster_size"] = gdf["cluster_label"].map(cluster_sizes)
    gdf.loc[gdf["cluster_label"] == -1, "cluster_size"] = 1
    gdf["cluster_score_n"] = minmax_series(gdf["cluster_size"]).values

    gdf["ndvi_loss_n"] = minmax_series(np.abs(np.minimum(gdf["ndvi_change"], 0))).values
    gdf["bsi_gain_n"] = minmax_series(np.maximum(gdf["bsi_change"], 0)).values
    gdf["ndmi_loss_n"] = minmax_series(np.abs(np.minimum(gdf["ndmi_change"], 0))).values
    gdf["mndwi_dist_n"] = minmax_series(np.abs(gdf["mndwi_change"])).values
    gdf["candidate_score_n"] = minmax_series(gdf["candidate_score"]).values

    feature_cols = ["ndvi_loss_n", "bsi_gain_n", "ndmi_loss_n", "mndwi_dist_n", "cluster_score_n"]
    weights, explained = compute_pca_weights(gdf, feature_cols)

    gdf["GDI_raw"] = 0.0
    for col, w in zip(feature_cols, weights):
        gdf["GDI_raw"] += w * gdf[col]

    gdf["GDI"] = minmax_series(gdf["GDI_raw"]).values

    thresholds = {
        "low_moderate": float(gdf["GDI"].quantile(LOW_MOD_Q)),
        "moderate_high": float(gdf["GDI"].quantile(MOD_HIGH_Q)),
        "high_critical": float(gdf["GDI"].quantile(HIGH_CRIT_Q)),
        "pc1_explained_variance": explained
    }

    gdf["GDI_Class"] = gdf["GDI"].apply(lambda x: classify_gdi_from_thresholds(
        x,
        thresholds["low_moderate"],
        thresholds["moderate_high"],
        thresholds["high_critical"]
    ))

    gdf["mining_prob"] = np.nanmean(
        np.stack([
            gdf["ndvi_loss_n"],
            gdf["bsi_gain_n"],
            gdf["ndmi_loss_n"],
            gdf["mndwi_dist_n"],
            gdf["candidate_score_n"]
        ]),
        axis=0
    )

    gdf["confidence"] = np.clip(2 * np.abs(gdf["mining_prob"] - 0.5), 0, 1)

    def decision_logic(row):
        if row["confidence"] < LOW_CONFIDENCE_THRESHOLD:
            return "LOW_CONFIDENCE"
        if row["legal_status"] == "LEGAL" and row["GDI_Class"] in ["Low", "Moderate"]:
            return "COMPLIANT"
        if row["legal_status"] == "LEGAL" and row["GDI_Class"] in ["High", "Critical"]:
            return "LEGAL_ANOMALY"
        if row["legal_status"] == "BOUNDARY":
            return "REVIEW_REQUIRED"
        if row["legal_status"] == "UNAUTHORIZED" and row["GDI_Class"] in ["High", "Critical"]:
            return "ILLEGAL_MINING"
        return "WATCHLIST"

    gdf["decision"] = gdf.apply(decision_logic, axis=1)
    gdf["lat"] = gdf.geometry.y
    gdf["lon"] = gdf.geometry.x
    gdf["lat_scaled"] = gdf["lat"].apply(scaled_coord)
    gdf["lon_scaled"] = gdf["lon"].apply(scaled_coord)
    gdf["GDI_scaled"] = gdf["GDI"].apply(scaled_gdi)

    weights_df = pd.DataFrame({"feature": feature_cols, "pca_weight": weights})
    weights_df["pc1 explained variance"] = explained

```

```

return gdf, thresholds, weights_df

def plot_temporal_comparison(ndvi_t1, ndvi_t2, ndvi_change, region_name):
    fig, axes = plt.subplots(1, 3, figsize=(18, 6))
    axes[0].imshow(ndvi_t1, cmap="RdYlGn", vmin=-1, vmax=1)
    axes[0].set_title(f"{region_name} NDVI {TIME_LABEL_1}")
    axes[0].axis("off")
    axes[1].imshow(ndvi_t2, cmap="RdYlGn", vmin=-1, vmax=1)
    axes[1].set_title(f"{region_name} NDVI {TIME_LABEL_2}")
    axes[1].axis("off")
    axes[2].imshow(ndvi_change, cmap="RdYlGn_r", vmin=-0.5, vmax=0.5)
    axes[2].set_title(f"{region_name} NDVI Change ({TIME_LABEL_2}-{TIME_LABEL_1})")
    axes[2].axis("off")
    plt.tight_layout()
    plt.savefig(os.path.join(OUTPUT_DIR, f"{region_name}_Temporal_Comparison.png"), dpi=300, bbox_inches="tight")
    plt.close()

def plot_multisignal_panels(ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, candidate_score, candidate_ma
fig, axes = plt.subplots(1, 7, figsize=(32, 5))
panels = [
    (ndvi_change, "NDVI Change", "RdYlGn_r", -0.5, 0.5),
    (bsi_change, "BSI Change", "YlOrBr", -0.5, 0.5),
    (ndmi_change, "NDMI Change", "BrBG", -0.5, 0.5),
    (mndwi_change, "MNDWI Change", "PuBu", -0.5, 0.5),
    (persistence_score, "Persistence", "viridis", 0, 1),
    (candidate_score, "Candidate Score", "magma", 0, 1),
    (candidate_mask, "Final Candidate Mask", "Reds", None, None)
]
for ax, (arr, title, cmap, vmin, vmax) in zip(axes, panels):
    ax.imshow(arr, cmap=cmap, vmin=vmin, vmax=vmax)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, f"{region_name}_MultiSignal_Candidate_Panels.png"), dpi=300, bbox_inches="tight")
plt.close()

def plot_risk_points_on_basemap(mining_gdf, region_name, bbox, output_png, concessions_gdf=None, label_points=False):
    aoi_poly = gpd.GeoDataFrame({"Region": [region_name], "geometry": [create_aoi_polygon(bbox)]}, crs="EPSG:4326")
    mining_web = mining_gdf.to_crs(epsg=3857)
    aoi_web = aoi_poly.to_crs(epsg=3857)

    fig, ax = plt.subplots(figsize=(14, 14))
    aoi_web.boundary.plot(ax=ax, edgecolor="cyan", linewidth=2)

    if SHOW_LEGAL_CONCESSION_ON_MAP and concessions_gdf is not None and not concessions_gdf.empty:
        concessions_gdf.to_crs(epsg=3857).boundary.plot(ax=ax, edgecolor="white", linestyle="--", linewidth=2)

    class_colors = {"Low": "lime", "Moderate": "yellow", "High": "orange", "Critical": "red"}
    for cls, color in class_colors.items():
        subset = mining_web[mining_web["GDI_Class"] == cls]
        if len(subset) > 0:
            subset.plot(ax=ax, color=color, markersize=18, alpha=0.85)

    if label_points:
        label_subset = mining_web[mining_web["GDI_Class"].isin(["High", "Critical"])].copy().reset_index(drop=True)
        label_latlon = mining_gdf[mining_gdf["GDI_Class"].isin(["High", "Critical"])].copy().reset_index(drop=True)
        for i, row in label_subset.iterrows():
            ax.text(
                row.geometry.x,
                row.geometry.y,
                f"{label_latlon.loc[i, 'lat']:.4f}, {label_latlon.loc[i, 'lon']:.4f}",
                fontsize=5,
                color="white",
                bbox=dict(facecolor="black", alpha=0.55, pad=0.3)
            )

    minx, miny, maxx, maxy = aoi_web.total_bounds
    ax.set_xlim(minx - (maxx - minx) * 0.05, maxx + (maxx - minx) * 0.05)
    ax.set_ylim(miny - (maxy - miny) * 0.05, maxy + (maxy - miny) * 0.05)

    ctx.add_basemap(ax, source=ctx.providers.Esri.WorldImagery, zoom="auto")
    try:
        ctx.add_basemap(ax, source=ctx.providers.CartoDB.VoyagerOnlyLabels, alpha=0.5)

```



```

except Exception:
    pass

legend_items = [Line2D([0], [0], color="cyan", lw=2, label="AOI")]
if SHOW_LEGAL_CONCESSION_ON_MAP and concessions_gdf is not None and not concessions_gdf.empty:
    legend_items.append(Line2D([0], [0], color="white", linestyle="--", lw=2, label="Legal concession"))
legend_items.extend([
    Line2D([0], [0], marker="o", color="w", markerfacecolor="lime", markersize=8, label="Low"),
    Line2D([0], [0], marker="o", color="w", markerfacecolor="yellow", markersize=8, label="Moderate"),
    Line2D([0], [0], marker="o", color="w", markerfacecolor="orange", markersize=8, label="High"),
    Line2D([0], [0], marker="o", color="w", markerfacecolor="red", markersize=8, label="Critical")
])
ax.legend(handles=legend_items, title="Layer / GDI Class", loc="lower left", framealpha=0.85)
ax.set_title(f"{region_name}: PCA-Weighted MNDWI-Enhanced GDI Risk Points")
ax.set_axis_off()
plt.tight_layout()
plt.savefig(output_png, dpi=300, bbox_inches="tight")
plt.close()

def add_evidence_hashes(mining_gdf, region_name):
    gdf = mining_gdf.copy()
    hashes = []
    for _, row in gdf.iterrows():
        payload = {
            "region": region_name,
            "lat_scaled": int(row["lat_scaled"]),
            "lon_scaled": int(row["lon_scaled"]),
            "gdi_scaled": int(row["GDI_scaled"]),
            "gdi_class": row["GDI_Class"],
            "legal_status": row["legal_status"],
            "decision": row["decision"],
            "source": SOURCE_NAME,
            "time_window_1": DATE_RANGE_1,
            "time_window_2": DATE_RANGE_2,
            "model": "NDVI_BSI_NDMI_MNDWI_Cluster_PCA_GDI"
        }
        hashes.append(compute_hash(payload))
    gdf["evidence_hash"] = hashes
    return gdf

def export_tables(mining_gdf, thresholds, weights_df, adaptive_thresholds_df, landcover_stats_df, region_name):
    mining_gdf = add_evidence_hashes(mining_gdf, region_name)

    class_counts = mining_gdf["GDI_Class"].value_counts().rename_axis("GDI_Class").reset_index(name="count")
    class_counts["percentage"] = (100 * class_counts["count"] / class_counts["count"].sum()).round(4)
    decision_counts = mining_gdf["decision"].value_counts().rename_axis("decision").reset_index(name="count")
    legal_counts = mining_gdf["legal_status"].value_counts().rename_axis("legal_status").reset_index(name="count")

    class_counts.to_csv(os.path.join(OUTPUT_DIR, "Summary.csv"), index=False)
    decision_counts.to_csv(os.path.join(OUTPUT_DIR, "Decision_Summary.csv"), index=False)
    legal_counts.to_csv(os.path.join(OUTPUT_DIR, "Legal_Status_Summary.csv"), index=False)

    with pd.ExcelWriter(os.path.join(OUTPUT_DIR, "Summary_Workbook.xlsx")) as writer:
        class_counts.to_excel(writer, sheet_name="GDI_Class", index=False)
        decision_counts.to_excel(writer, sheet_name="Decision", index=False)
        legal_counts.to_excel(writer, sheet_name="Legal_Status", index=False)

    pd.DataFrame([thresholds]).to_csv(os.path.join(OUTPUT_DIR, f"Adaptive_GDI_Thresholds_{region_name}.csv"), index=False)
    weights_df.to_csv(os.path.join(OUTPUT_DIR, f"PCA_GDI_Weights_{region_name}.csv"), index=False)
    adaptive_thresholds_df.to_csv(os.path.join(OUTPUT_DIR, f"Adaptive_Thresholds_{region_name}.csv"), index=False)
    landcover_stats_df.to_csv(os.path.join(OUTPUT_DIR, f"Landcover_Masking_Stats_{region_name}.csv"), index=False)

    internal = mining_gdf.groupby("GDI_Class").agg(
        Count=("GDI", "count"),
        Mean_GDI=("GDI", "mean"),
        Mean_NDVI_Change=("ndvi_change", "mean"),
        Mean_BSI_Change=("bsi_change", "mean"),
        Mean_NDMI_Change=("ndmi_change", "mean"),
        Mean_MNDWI_Change=("mndwi_change", "mean"),
        Mean_Persistence=("persistence_score", "mean"),
        Mean_Candidate_Score=("candidate_score", "mean")
    ).round(6).reset_index()
    internal.to_csv(os.path.join(OUTPUT_DIR, f"Internal_Class_Separation_{region_name}.csv"), index=False)

    mining_gdf.to_csv(os.path.join(OUTPUT_DIR, f"Mining_Points_{region_name}.csv"), index=False)
    try:

```

```

        mining_gdf.to_file(os.path.join(OUTPUT_DIR, f"Mining_Points_{region_name}.shp"))
    except Exception as e:
        print(f"Shapefile export skipped/failed: {e}")

    coord_cols = [
        "Region", "lat", "lon", "lat_scaled", "lon_scaled", "GDI", "GDI_scaled", "GDI_Class",
        "legal_status", "decision", "evidence_hash"
    ]
    mining_gdf[ mining_gdf["GDI_Class"] == "Moderate" ][coord_cols].to_csv(
        os.path.join(OUTPUT_DIR, f"Moderate_Risk_Coordinates_{region_name}.csv"), index=False
    )
    mining_gdf[ mining_gdf["GDI_Class"].isin(["High", "Critical"]) ][coord_cols].to_csv(
        os.path.join(OUTPUT_DIR, f"High_Critical_Risk_Coordinates_{region_name}.csv"), index=False
    )

    risk = mining_gdf[ mining_gdf["GDI_Class"].isin(["Moderate", "High", "Critical"]) ].copy()
    enum_map = {"Low": 0, "Moderate": 1, "High": 2, "Critical": 3}
    evidence_records = []
    for _, row in risk.iterrows():
        tuple_input = [
            1,
            int(row["lat_scaled"]),
            int(row["lon_scaled"]),
            int(row["GDI_scaled"]),
            enum_map[row["GDI_Class"]],
            row["legal_status"],
            row["decision"],
            row["evidence_hash"],
            ANALYSIS_RESULT_CID
        ]
        evidence_records.append({
            "record_id": len(evidence_records) + 1,
            "Region": region_name,
            "lat": row["lat"],
            "lon": row["lon"],
            "lat_scaled": int(row["lat_scaled"]),
            "lon_scaled": int(row["lon_scaled"]),
            "GDI": row["GDI"],
            "GDI_scaled": int(row["GDI_scaled"]),
            "GDI_Class": row["GDI_Class"],
            "riskClass_enum": enum_map[row["GDI_Class"]],
            "legal_status": row["legal_status"],
            "decision": row["decision"],
            "analysisResultCID": ANALYSIS_RESULT_CID,
            "evidence_hash": row["evidence_hash"],
            "submitCoordinateGDIRecord_tuple": str(tuple_input)
        })
    evidence_df = pd.DataFrame(evidence_records)
    evidence_df.to_csv(os.path.join(OUTPUT_DIR, f"Blockchain_Ready_Risk_Evidence_{region_name}.csv"), index=False)

    oracle = mining_gdf[ mining_gdf["decision"] == "ILLEGAL_MINING" ].copy()
    oracle_cols = ["lat", "lon", "lat_scaled", "lon_scaled", "GDI", "GDI_scaled", "GDI_Class", "legal_status", "decision", "evidence_hash"]
    oracle[oracle_cols].to_csv(os.path.join(OUTPUT_DIR, f"Oracle_{region_name}.csv"), index=False)

    return mining_gdf, evidence_df

def simulate_supply_chain_transactions(evidence_df, n_transactions=20, seed=42):
    np.random.seed(seed)
    if evidence_df.empty:
        return pd.DataFrame()

    actors = {
        "asm": ["ASM_01", "ASM_02", "ASM_03", "ASM_04"],
        "processor": ["PROC_01", "PROC_02"],
        "trader": ["TRAD_01", "TRAD_02"],
        "assayer": ["ASSAY_01"],
        "bullion_bank": ["BANK_01"]
    }

    sampled = evidence_df.sample(n=min(n_transactions, len(evidence_df)), replace=True, random_state=seed).reset_index(drop=True)
    rows = []
    for i, row in sampled.iterrows():
        rows.append({
            "Transaction_ID": f"TX_{i+1:04d}",
            "Gold_Batch_ID": f"BATCH_OBUASI_{i+1:04d}",
            "Source_Record_ID": int(row["record_id"]),

```

```

        "Source_Region": row["Region"],
        "Source_lat": round(float(row["lat"]), 6),
        "Source_lon": round(float(row["lon"]), 6),
        "Source_lat_scaled": int(row["lat_scaled"]),
        "Source_lon_scaled": int(row["lon_scaled"]),
        "Source_GDI": round(float(row["GDI"]), 6),
        "Source_GDI_scaled": int(row["GDI_scaled"]),
        "Source_GDI_Class": row["GDI_Class"],
        "Legal_Status": row["legal_status"],
        "Decision_Basis": row["decision"],
        "Evidence_Hash": row["evidence_hash"],
        "Origin_Actor": np.random.choice(actors["asm"]),
        "Processor": np.random.choice(actors["processor"]),
        "Trader": np.random.choice(actors["trader"]),
        "Assayer": np.random.choice(actors["assayer"]),
        "Bullion_Bank": np.random.choice(actors["bullion_bank"]),
        "Declared_Certified": np.random.choice([True, False], p=[0.70, 0.30]),
        "Declared_Legal_Origin": np.random.choice([True, False], p=[0.75, 0.25]),
        "Quantity_kg": round(np.random.uniform(2, 25), 2)
    })
    return pd.DataFrame(rows)

def validate_transaction(tx_row):
    gdi_class = tx_row["Source_GDI_Class"]
    certified = tx_row["Declared_Certified"]
    declared_legal = tx_row["Declared_Legal_Origin"]
    decision_basis = tx_row["Decision_Basis"]

    if gdi_class == "Critical" or decision_basis == "ILLEGAL_MINING":
        return "REJECTED", "Critical or illegal-mining source zone"
    if gdi_class == "High":
        return "REJECTED", "High-risk source zone"
    if gdi_class == "Moderate" and not certified:
        return "REJECTED", "Moderate-risk zone with missing certification"
    if not certified:
        return "REJECTED", "Missing certification"
    if not declared_legal:
        return "FLAGGED", "Suspicious legal-origin declaration"
    return "APPROVED", "Compliant transaction"

def run_transaction_screening(evidence_df):
    tx_df = simulate_supply_chain_transactions(evidence_df, n_transactions=20, seed=RANDOM_SEED)
    if tx_df.empty:
        tx_df.to_csv(os.path.join(OUTPUT_DIR, "Supply_Chain_Transactions_Screened.csv"), index=False)
        return tx_df

    statuses = tx_df.apply(validate_transaction, axis=1)
    tx_df["Decision"] = [s[0] for s in statuses]
    tx_df["Decision_Reason"] = [s[1] for s in statuses]

    tx_df["updateGoldBatchTransaction_tuple"] = tx_df.apply(
        lambda r: str([
            r["Gold_Batch_ID"],
            int(r["Source_Record_ID"]),
            int(r["Source_lat_scaled"]),
            int(r["Source_lon_scaled"]),
            int(r["Source_GDI_scaled"]),
            int(round(float(r["Quantity_kg"]) * 1000)),
            bool(r["Declared_Certified"]),
            bool(r["Declared_Legal_Origin"]),
            r["Evidence_Hash"]
        ]), axis=1
    )

    tx_df.to_csv(os.path.join(OUTPUT_DIR, "Supply_Chain_Transactions_Screened.csv"), index=False)
    tx_summary = tx_df["Decision"].value_counts().rename_axis("Transaction_Status").reset_index(name="count")
    tx_summary.to_csv(os.path.join(OUTPUT_DIR, "Supply_Chain_Transaction_Summary.csv"), index=False)
    return tx_df

def process_candidates_for_percentile(percentile, ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, ref_meta:
    ndvi_thr, bsi_thr, ndmi_thr, mndwi_thr = derive_adaptive_thresholds(ndvi_change, bsi_change, ndmi_change, mndwi_change, per
    cands, _, _ = extract_candidate_points(
        ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, ref_meta, region_name,
        ndvi_thr, bsi_thr, ndmi_thr, mndwi_thr

```

```

    cands, _ = apply_optional_landcover_mask(cands)
    cands = apply_legal_screening(cands, concessions_gdf)
    gdf, thresholds, weights_df = compute_gdi(cands)
    gdf = add_evidence_hashes(gdf, region_name) if not gdf.empty else gdf
    return gdf, thresholds, weights_df, {
        "percentile": percentile,
        "NDVI_loss_threshold": ndvi_thr,
        "BSI_gain_threshold": bsi_thr,
        "NDMI_loss_threshold": ndmi_thr,
        "MNDWI_dist_threshold": mndwi_thr
    }
}

def run_threshold_sensitivity(ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, ref_meta, region_name, concessions_gdf):
    records = []
    hotspot_sets = {}
    for pct in SENSITIVITY_PERCENTILES:
        gdf, thresholds, _, spectral_thr = process_candidates_for_percentile(
            pct, ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, ref_meta, region_name, concessions_gdf
        )
        if gdf.empty:
            records.append({"percentile": pct, "candidate_points": 0, "high_critical_points": 0})
            hotspot_sets[pct] = set()
            continue

        highcrit = gdf[gdf["GDI_Class"].isin(["High", "Critical"])]
        hotspot_sets[pct] = set(zip(highcrit["lat_scaled"], highcrit["lon_scaled"]))
        records.append({
            "percentile": pct,
            "NDVI_loss_threshold": spectral_thr["NDVI_loss_threshold"],
            "BSI_gain_threshold": spectral_thr["BSI_gain_threshold"],
            "NDMI_loss_threshold": spectral_thr["NDMI_loss_threshold"],
            "MNDWI_dist_threshold": spectral_thr["MNDWI_dist_threshold"],
            "candidate_points": len(gdf),
            "low_points": int((gdf["GDI_Class"] == "Low").sum()),
            "moderate_points": int((gdf["GDI_Class"] == "Moderate").sum()),
            "high_points": int((gdf["GDI_Class"] == "High").sum()),
            "critical_points": int((gdf["GDI_Class"] == "Critical").sum()),
            "high_critical_points": len(highcrit),
            "illegal_mining_points": int((gdf["decision"] == "ILLEGAL_MINING").sum()),
            "pc1_explained_variance": thresholds.get("pc1_explained_variance", np.nan)
        })

    pd.DataFrame(records).to_csv(os.path.join(OUTPUT_DIR, "Threshold_Sensitivity_Summary.csv"), index=False)

    overlap_records = []
    for i, p1 in enumerate(SENSITIVITY_PERCENTILES):
        for p2 in SENSITIVITY_PERCENTILES[i+1:]:
            s1, s2 = hotspot_sets[p1], hotspot_sets[p2]
            union = len(s1.union(s2))
            inter = len(s1.intersection(s2))
            overlap_records.append({
                "percentile_a": p1,
                "percentile_b": p2,
                "hotspot_overlap_count": inter,
                "hotspot_union_count": union,
                "jaccard_overlap": inter / union if union else np.nan
            })
    pd.DataFrame(overlap_records).to_csv(os.path.join(OUTPUT_DIR, "Threshold_Sensitivity_Hotspot_Overlap.csv"), index=False)

def run_concession_buffer_sensitivity(candidates_gdf, concessions_gdf):
    records = []
    for buffer_degrees in [0.001, 0.002, 0.003, 0.005]:
        screened = apply_legal_screening(candidates_gdf, concessions_gdf, buffer_degrees=buffer_degrees)
        counts = screened["legal_status"].value_counts().to_dict() if not screened.empty else {}
        records.append({
            "buffer_degrees": buffer_degrees,
            "LEGAL": counts.get("LEGAL", 0),
            "BOUNDARY": counts.get("BOUNDARY", 0),
            "UNAUTHORIZED": counts.get("UNAUTHORIZED", 0)
        })
    pd.DataFrame(records).to_csv(os.path.join(OUTPUT_DIR, "Concession_Buffer_Sensitivity.csv"), index=False)

def process_region(region_name, bbox, concessions_gdf):
    print("\n" + "=" * 80)

```

```

print(f"PROCESSING {region_name.upper()}")
print("=" * 80)

aoi_geojson = bbox_to_geojson(bbox)
aoi_polygon = create_aoi_polygon(bbox)

item_t1 = search_best_scene(DATE_RANGE_1, TIME_LABEL_1, aoi_geojson)
item_t2 = search_best_scene(DATE_RANGE_2, TIME_LABEL_2, aoi_geojson)

blue_t1, blue_meta_t1 = read_clip_band(item_t1, "B02", aoi_polygon)
green_t1, green_meta_t1 = read_clip_band(item_t1, "B03", aoi_polygon)
red_t1, red_meta_t1 = read_clip_band(item_t1, "B04", aoi_polygon)
nir_t1, nir_meta_t1 = read_clip_band(item_t1, "B08", aoi_polygon)
swir_t1, swir_meta_t1 = read_clip_band(item_t1, "B11", aoi_polygon)

blue_t2, blue_meta_t2 = read_clip_band(item_t2, "B02", aoi_polygon)
green_t2, green_meta_t2 = read_clip_band(item_t2, "B03", aoi_polygon)
red_t2, red_meta_t2 = read_clip_band(item_t2, "B04", aoi_polygon)
nir_t2, nir_meta_t2 = read_clip_band(item_t2, "B08", aoi_polygon)
swir_t2, swir_meta_t2 = read_clip_band(item_t2, "B11", aoi_polygon)

ref_meta = red_meta_t1.copy()

blue_t1 = align_to_reference(blue_t1, blue_meta_t1, ref_meta)
green_t1 = align_to_reference(green_t1, green_meta_t1, ref_meta)
red_t1 = align_to_reference(red_t1, red_meta_t1, ref_meta)
nir_t1 = align_to_reference(nir_t1, nir_meta_t1, ref_meta)
swir_t1 = align_to_reference(swir_t1, swir_meta_t1, ref_meta)

blue_t2 = align_to_reference(blue_t2, blue_meta_t2, ref_meta)
green_t2 = align_to_reference(green_t2, green_meta_t2, ref_meta)
red_t2 = align_to_reference(red_t2, red_meta_t2, ref_meta)
nir_t2 = align_to_reference(nir_t2, nir_meta_t2, ref_meta)
swir_t2 = align_to_reference(swir_t2, swir_meta_t2, ref_meta)

ndvi_t1 = compute_ndvi(nir_t1, red_t1)
ndvi_t2 = compute_ndvi(nir_t2, red_t2)
ndvi_change = ndvi_t2 - ndvi_t1

bsi_t1 = compute_bsi(red_t1, nir_t1, blue_t1, swir_t1)
bsi_t2 = compute_bsi(red_t2, nir_t2, blue_t2, swir_t2)
bsi_change = bsi_t2 - bsi_t1

ndmi_t1 = compute_ndmi(nir_t1, swir_t1)
ndmi_t2 = compute_ndmi(nir_t2, swir_t2)
ndmi_change = ndmi_t2 - ndmi_t1

mndwi_t1 = compute_mndwi(green_t1, swir_t1)
mndwi_t2 = compute_mndwi(green_t2, swir_t2)
mndwi_change = mndwi_t2 - mndwi_t1

# Save raster figures
save_array_png(ndvi_t1, f"{region_name} NDVI {TIME_LABEL_1}", os.path.join(OUTPUT_DIR, f"{region_name}_NDVI_{TIME_LABEL_1}.png"))
save_array_png(ndvi_t2, f"{region_name} NDVI {TIME_LABEL_2}", os.path.join(OUTPUT_DIR, f"{region_name}_NDVI_{TIME_LABEL_2}.png"))
save_array_png(ndvi_change, f"{region_name} NDVI Change", os.path.join(OUTPUT_DIR, f"{region_name}_NDVI_Change.png"), cmap="magma")
save_array_png(bsi_change, f"{region_name} BSI Change", os.path.join(OUTPUT_DIR, f"{region_name}_BSI_Change.png"), cmap="magma")
save_array_png(ndmi_change, f"{region_name} NDMI Change", os.path.join(OUTPUT_DIR, f"{region_name}_NDMI_Change.png"), cmap="magma")
save_array_png(mndwi_change, f"{region_name} MNDWI Change", os.path.join(OUTPUT_DIR, f"{region_name}_MNDWI_Change.png"), cmap="magma")
plot_temporal_comparison(ndvi_t1, ndvi_t2, ndvi_change, region_name)

ndvi_thr, bsi_thr, ndmi_thr, mndwi_thr = derive_adaptive_thresholds(
    ndvi_change, bsi_change, ndmi_change, mndwi_change, ADAPTIVE_PERCENTILE
)
adaptive_thresholds_df = pd.DataFrame([
    {
        "adaptive_percentile": ADAPTIVE_PERCENTILE,
        "NDVI_loss_threshold": ndvi_thr,
        "BSI_gain_threshold": bsi_thr,
        "NDMI_loss_threshold": ndmi_thr,
        "MNDWI_dist_threshold": mndwi_thr,
        "persistence_threshold": PERSISTENCE_THRESHOLD,
        "candidate_score_threshold": CANDIDATE_SCORE_THRESHOLD
    }
])
print(f"Adaptive thresholds: NDVI={ndvi_thr:.4f}, BSI={bsi_thr:.4f}, NDMI={ndmi_thr:.4f}, MNDWI={mndwi_thr:.4f}")

persistence_score = compute_persistence_score(ndvi_change, bsi_change, ndmi_change, mndwi_change)
save_array_png(persistence_score, f"{region_name} Persistence Score", os.path.join(OUTPUT_DIR, f"{region_name}_Persistence_Score.png"))

```

```

candidates, candidate_score, candidate_mask = extract_candidate_points(
    ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, ref_meta, region_name,
    ndvi_thr, bsi_thr, ndmi_thr, mndwi_thr
)
print(f"Candidate disturbance points extracted before land-cover mask: {len(candidates)}")

plot_multisignal_panels(ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, candidate_score, candidate_ma

candidates, landcover_stats_df = apply_optional_landcover_mask(candidates)
print(f"Candidate disturbance points after land-cover mask: {len(candidates)}")

if candidates.empty:
    print("No candidate points detected. Pipeline finished.")
    return None

run_concession_buffer_sensitivity(candidates, concessions_gdf)

candidates = apply_legal_screening(candidates, concessions_gdf)
mining_gdf, thresholds, weights_df = compute_gdi(candidates)

mining_gdf, evidence_df = export_tables(mining_gdf, thresholds, weights_df, adaptive_thresholds_df, landcover_stats_df, regio

plot_risk_points_on_basemap(
    mining_gdf, region_name, bbox,
    os.path.join(OUTPUT_DIR, f"{region_name}_Risk_Basemap.png"),
    concessions_gdf=concessions_gdf,
    label_points=False
)
plot_risk_points_on_basemap(
    mining_gdf, region_name, bbox,
    os.path.join(OUTPUT_DIR, f"{region_name}_HighRisk_Coordinates_Map.png"),
    concessions_gdf=concessions_gdf,
    label_points=SHOW_HIGH_RISK_LABELS_ON_MAP
)

run_threshold_sensitivity(ndvi_change, bsi_change, ndmi_change, mndwi_change, persistence_score, ref_meta, region_name, conce
tx_df = run_transaction_screening(evidence_df)

print("\n==== FINAL SUMMARY =====")
print(mining_gdf["GDI_Class"].value_counts())
print("\nDecision summary:")
print(mining_gdf["decision"].value_counts())
print("\nTransaction summary:")
if not tx_df.empty:
    print(tx_df["Decision"].value_counts())
print(f"\nFULL PIPELINE COMPLETE. Outputs stored in: {OUTPUT_DIR}")

return mining_gdf

def main():
    process_region(AOI_NAME, AOI_BBOX, concessions)

if __name__ == "__main__":
    main()

```

```

===== 41.8/41.8 kB 2.5 MB/s eta 0:00:00
===== 208.5/208.5 kB 12.1 MB/s eta 0:00:00

Configuration loaded.
Connected to Microsoft Planetary Computer.
Concession file not found.
Using demo legal zone for external governance screening.

=====
PROCESSING OBUASI
=====
Searching Sentinel-2 scene for Jan_2026: 2026-01-01/2026-01-31
Selected Jan_2026: S2B_MSIL2A_20260125T102219_R065_T30NXN_20260125T155454, cloud=0.023557
Searching Sentinel-2 scene for Apr_2026: 2026-02-01/2026-04-30
Selected Apr_2026: S2C_MSIL2A_20260430T102021_R065_T30NXN_20260430T164214, cloud=4.795827
Adaptive thresholds: NDVI=-0.0967, BSI=0.0624, NDMI=-0.0837, MNDWI=0.0432
Candidate disturbance points extracted before land-cover mask: 1161
Candidate disturbance points after land-cover mask: 1161

===== FINAL SUMMARY =====
GDI_Class

```

```
Low          580
Moderate     290
High         174
Critical     117
Name: count, dtype: int64
```

Decision summary:

```
decision
LOW_CONFIDENCE    560
WATCHLIST         397
COMPLIANT         104
LEGAL_ANOMALY      77
REVIEW_REQUIRED   16
ILLEGAL_MINING      7
Name: count, dtype: int64
```

Transaction summary:

```
Decision
REJECTED    11
APPROVED     7
FLAGGED      2
Name: count, dtype: int64
```

FULL PIPELINE COMPLETE. Outputs stored in: Obuassi_GDI_Analysis

Start coding or [generate](#) with AI.